Complete React.js Concepts & Hooks

Interview preparation for a language-learning platform (LSRW)

Scope: React fundamentals through advanced patterns, every built-in React hook in the current React documentation, custom hooks, common ecosystem topics, practical LSRW examples, and interview recall prompts.

Honesty rule: this is a study guide, not evidence of what learn.lingotran.com actually implemented. Mark each feature Used / Possibly used / Not used or unsure. Do not claim a library or technique was used unless you remember or can verify it.

How to use this guide

For each topic: (1) explain it in your own words, (2) recall a screen or bug from your project, (3) write a tiny example, (4) practise a 30–60 second interview answer, and (5) distinguish what you actually built from what you would build today.

1. React foundations

What React is

A JavaScript library for building user interfaces from components. React describes UI from state and props; it does not prescribe your entire backend or routing stack.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Declarative UI

Describe what the interface should look like for current data; React updates the rendered UI when data changes.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Components

Reusable functions that return JSX. Keep components focused; compose small pieces into larger screens.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

JSX

Syntax extension that looks like HTML but follows JavaScript rules: use className, camelCase event names, one parent/root or a fragment, and braces for expressions.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Fragments

<>... groups siblings without adding an extra DOM element.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Props

Read-only inputs passed from parent to child. Destructure props as convenient; do not mutate them.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

State

Component-owned changing data. A state setter requests a new render; setting the same value may be skipped.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Render and commit

React calls components to calculate UI, then commits necessary DOM changes. Render functions should be pure.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Keys

Stable identities for items in lists. Prefer database IDs or stable IDs over array indexes for reorderable lists.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Conditional rendering

Use if/else, ternary, && or early returns to display loading, empty, error, activity and result states.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Event handling

Pass handlers such as onClick={handleClick}; do not call the handler during render unless calling it is intended.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Controlled inputs

Input value is driven by React state; update it in onChange. Useful for writing answers, forms and search.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Uncontrolled inputs

DOM maintains the current value; use refs or FormData when appropriate. Choose deliberately rather than mixing patterns accidentally.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Composition

Build features by nesting components and passing children/slots instead of creating giant components with many flags.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Lifting state

Move shared state to the nearest common parent when sibling components need to coordinate.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Derived data

Calculate values from props/state during render when possible; avoid duplicating derived state in an effect.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Immutability

Create new arrays/objects instead of mutating existing state. This helps React detect changes predictably.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

Purity and Strict Mode

Components and updater functions should be pure. Development Strict Mode may re-run some logic to expose unsafe side effects; it is not itself a production double-render guarantee.

Recall: [ ] Used [ ] Possibly used [ ] Not used / unsure

2. All built-in React hooks: quick reference

Hooks must be called at the top level of function components or custom hooks—not inside loops, conditions or nested functions. The list below covers the built-in hooks documented by React; availability can vary by React version and framework.

useState

Local state for values that change over time. Example: selected answer, form text, current question, play/pause state.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useReducer

State transitions described by a reducer and actions. Useful for a multi-step activity with related states and explicit transitions.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useContext

Read a context value supplied by a matching provider above the component. Useful for shared theme, locale or authenticated-user context when appropriate.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useRef

Persist a mutable value or DOM reference across renders without causing a render when changed. Example: focus an input or control an audio element.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useEffect

Synchronise with an external system after rendering: subscriptions, browser APIs or network-related integration. Do not use it for every derived calculation.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useLayoutEffect

Run an effect after DOM changes but before the browser paints; use for layout measurement/positioning when a visible flicker must be avoided.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useInsertionEffect

Primarily for CSS-in-JS library authors to insert styles before layout effects. Most application code should not need it.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useMemo

Cache a calculation result between renders when worthwhile. A performance optimisation, not a correctness mechanism or general storage.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useCallback

Cache a function definition between renders. Useful when function identity matters to a memoized child or another hook.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useTransition

Mark some state updates as non-urgent so urgent interactions can stay responsive. Does not make the underlying work intrinsically faster.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useDeferredValue

Let a part of the UI lag behind a rapidly changing value, often to keep typing responsive while expensive UI updates.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useId

Generate stable IDs for accessibility relationships such as label/input or aria-describedby. Do not use it for list keys.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useSyncExternalStore

Subscribe to an external store with a snapshot and subscription function; useful for store/library integration.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useDebugValue

Display a label for a custom hook in React DevTools. Mainly useful when authoring reusable hooks.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useActionState

Manage state associated with an Action/form submission and its returned result in React versions that support it.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useOptimistic

Show an optimistic UI state while an Action is pending, then reconcile with the result.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

use

Read a resource such as a Promise or context in supported React contexts. Unlike ordinary hooks, it may be used conditionally and in loops where documented; it still has restrictions and must be used within a component or hook.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

useEffectEvent

Define an Effect Event to read the latest props/state from effect logic without making that logic reactive in the same way as the effect. Follow current React-version guidance; do not use as a general event-handler replacement.

Project recall: Which real screen, if any, makes this familiar? [ ] Used [ ] Maybe [ ] Unsure

3. Hook rules and common mistakes

Hook order

Call hooks at the top level of a function component or custom hook. Never conditionally call useState/useEffect or call them in loops.

Effect dependencies

List reactive values used by the effect. Do not silence dependency warnings without understanding the stale-value or loop risk.

Effect cleanup

Remove listeners, unsubscribe, stop timers or ignore/cancel outdated work when the effect is cleaned up.

Avoid effect loops

An effect that updates state it depends on may repeatedly run. Ask whether the value can be calculated during render or the update can be handled in an event.

State snapshots

A handler sees the state snapshot for its render. Use functional updates such as setCount(c => c + 1) when the next value depends on the previous one.

Refs versus state

Changing ref.current does not trigger rendering. Use state if the UI must update to reflect a value.

Memoisation

useMemo/useCallback/React.memo are optimisations. First keep state local, render pure components and measure actual bottlenecks.

Async effects

An effect callback itself should not be async because effects may return cleanup functions. Define an async function inside it or call a separate function; handle cancellation/stale responses.

Strict Mode

Extra development checks may reveal missing cleanup or impure render logic. Avoid “fixes” that merely hide the checks.

Server/client constraints

Some frameworks distinguish server and client components. Hooks that rely on client state/effects belong in supported client components.

4. Custom hooks and reusable logic

A custom hook is a JavaScript function whose name starts with use and that may call other hooks. It shares stateful logic—not one shared state instance by default—between components.

Possible LSRW-oriented custom hooks (only claim these if you built them):

useLesson(lessonId)

Fetch lesson details and expose data/loading/error/refetch.

useAudioPlayer()

Coordinate audio element refs, playback state and media events.

useWritingDraft()

Manage draft text, word count, validation and saved/submitted status.

useActivityProgress()

Load or update server-backed progress and expose pending/error states.

useDebouncedValue(value, delay)

Return a delayed value for search or filtering; use a proper timer cleanup.

Custom-hook design tips

Keep each hook focused; expose a clear return shape; handle loading/error states; clean up subscriptions; avoid hiding too much behaviour; and remember that each call has its own hook state unless it connects to a shared external store/context.

5. React ecosystem concepts to know

Routing

React Router or a framework router maps URLs to screens, supports parameters and navigation, and may support nested/protected routes. Confirm the actual library used.

Global state

Context, Redux, Zustand or another store can share client state. Choose based on actual complexity; server data is often better managed separately.

Server state / TanStack Query

Can handle query caching, refetching, mutations, pending/error states and invalidation. Only claim project use if remembered.

Forms and validation

Native form handling or libraries such as React Hook Form can manage values and validation. Server-side validation remains necessary.

API clients

fetch or axios may call Node endpoints. A service module can centralise URLs, headers, error parsing and request logic.

Lazy loading and Suspense

React.lazy can load component code on demand; Suspense shows fallback UI for supported suspended work.

Error boundaries

Catch rendering errors in a subtree and show fallback UI. They do not automatically catch every event-handler or asynchronous error.

Portals

Render UI such as modals outside the parent DOM hierarchy while keeping React ownership.

Refs and imperative handles

forwardRef is relevant to older/current library patterns; useImperativeHandle exposes a limited imperative API from a child.

Testing

Test user-visible behaviour, state transitions, forms, loading/error states and accessibility. Tooling might include React Testing Library and an end-to-end test runner.

Performance

Use React DevTools Profiler, split components, keep state local, virtualise very long lists if needed, and memoize only where there is evidence.

Accessibility

Semantic HTML, keyboard operation, labels, focus management, transcripts/captions and understandable errors are especially relevant to learning activities.

6. LSRW feature mapping: React patterns to recall

Listening

Audio player; play/pause/replay; playback progress; audio-ended event; question/answer state; loading/error state. Recall whether you used an HTML audio element, ref, media events, or a player library.

Actual project status: [ ] Used [ ] Possibly used [ ] Not used / unsure

Speaking

Start/stop recording UI; permission status; audio upload; transcription or pronunciation feedback. Browser MediaRecorder/Web Speech APIs or server-side services are possibilities, not assumptions.

Actual project status: [ ] Used [ ] Possibly used [ ] Not used / unsure

Reading

Passage and vocabulary display; question list; selected answer; highlight state; comprehension feedback; lesson API data.

Actual project status: [ ] Used [ ] Possibly used [ ] Not used / unsure

Writing

Controlled textarea; draft state; word count; validation; submit action; pending/error/success UI; optional feedback returned by an API.

Actual project status: [ ] Used [ ] Possibly used [ ] Not used / unsure

Progress

Completed lesson indicators, scores and progress bars. Recall whether data came from the server, local state, Context or a store.

Actual project status: [ ] Used [ ] Possibly used [ ] Not used / unsure

Across all skills

Reusable lesson shell, consistent buttons, responsive layout, API error handling, accessibility and tracking of user progress.

Actual project status: [ ] Used [ ] Possibly used [ ] Not used / unsure

7. Interview questions and answer structure

Explain React.

A JavaScript library for building component-based user interfaces. Explain the screens you personally worked on.

Props vs state?

Props are inputs from a parent; state is data managed by a component and updated through setters/reducers.

Why useEffect?

To synchronise with external systems. Give a true example such as a subscription, event listener or data-loading flow.

useMemo vs useCallback?

useMemo caches a computed value; useCallback caches a function definition. Both are performance optimisations.

useRef vs useState?

A ref persists a mutable value without triggering a render; state updates the UI by requesting a render.

How did you handle API data?

Describe the actual trigger, request, loading/success/error states, response handling and what happened on failure.

How would you design a listening activity?

Describe components, media controls, activity state, question flow, API boundaries, accessibility and failure cases. If hypothetical, say “I would design…”

How do you avoid unnecessary renders?

Keep state local, avoid needless state duplication, keep render pure, split components sensibly, profile, and memoize where justified.

What did you personally deliver?

Use a specific screen, component, bug fix or API integration you genuinely remember; distinguish team work from your contribution.

What if you don't remember exact implementation?

Be transparent: “I don't want to guess the exact library/code. The problem was X; the standard approach is Y; I'd verify the original implementation before claiming that detail.”

8. Hands-on exercises

1. [ ] Build a LessonCard receiving title, level and completion status as props.
2. [ ] Render a list of lesson cards using stable IDs as keys.
3. [ ] Build a multiple-choice question with useState and correct/incorrect feedback.
4. [ ] Build a writing form with controlled textarea, word count and validation.
5. [ ] Load mock lesson data and show loading, error, empty and success states.
6. [ ] Use useRef to focus an input after a button click.
7. [ ] Build an audio-player UI using a real audio element and media events.
8. [ ] Write a custom hook for a small, cancellable data request or subscription.
9. [ ] Use useReducer for an activity with idle, answering, submitted and completed states.
10. [ ] Add accessible labels, keyboard support and clear error messages.

9. Official documentation and version notes

React Learn: react.dev/learn

Built-in Hooks: react.dev/reference/react/hooks

React API reference: react.dev/reference/react

Rules of Hooks: react.dev/reference/rules

> React adds and evolves APIs over time. Confirm the React version used by a project before describing version-specific hooks such as useActionState, useOptimistic, useEffectEvent or use.

Revision checklist

[ ] I can explain every hook in one or two sentences. [ ] I can write a tiny example. [ ] I know its common misuse. [ ] I can connect it to a real feature I remember. [ ] I can distinguish actual experience from a hypothetical design.